Lecture 28: Practice Problems Master Solutions & Systems Manual

Supervised Machine Learning (Part 4) — Decision Trees

Complete mathematical proofs, verified production code implementations, and case study evaluations.

Problem 01: Pure NumPy Shannon Entropy Calculator (ShannonEntropyCalculator)

💡 Production Python Implementation

```
import numpy as np

class ShannonEntropyCalculator:
    """
    Pure NumPy implementation of Claude Shannon's Information Entropy for node imp
    Formula: H(S) = - sum(p_k * log2(p_k)) with 0 * log2(0) = 0 convention.
    """
    @staticmethod
    def compute(labels):
        labels = np.asarray(labels)
        if len(labels) == 0:
            return 0.0
        _, counts = np.unique(labels, return_counts=True)
        probs = counts / counts.sum()
        # Enforce 0 * log2(0) = 0 convention
        non_zero_probs = probs[probs > 0]
        return float(-np.sum(non_zero_probs * np.log2(non_zero_probs)))

# Unit Test
calc = ShannonEntropyCalculator()
assert calc.compute([1, 1, 1, 1]) == 0.0, 'Pure node entropy must be 0.0'
assert round(calc.compute([0, 1, 0, 1]), 4) == 1.0, '50-50 mix entropy must be 1.0
assert round(calc.compute([0, 1, 2]), 4) == round(np.log2(3), 4), 'Uniform 3-class
print('Problem 01 Passed!')
```

Problem 02: Pure NumPy Gini Impurity & Weighted Split Evaluator (GiniImpurityEvaluator)

💡 Production Python Implementation

```
import numpy as np

class GiniImpurityEvaluator:
    """
    Pure NumPy implementation of Gini Impurity and weighted split impurity evaluat
    """
    @staticmethod
    def compute_node(labels):
        labels = np.asarray(labels)
        if len(labels) == 0:
            return 0.0
        _, counts = np.unique(labels, return_counts=True)
        probs = counts / counts.sum()
        return float(1.0 - np.sum(probs ** 2))

    @classmethod
    def evaluate_split(cls, parent_labels, left_labels, right_labels):
        n_p = len(parent_labels)
        n_l = len(left_labels)
        n_r = len(right_labels)
        if n_p == 0 or n_l == 0 or n_r == 0:
            return 0.0, 0.0 # No split possible

        g_parent = cls.compute_node(parent_labels)
        g_children = (n_l / n_p) * cls.compute_node(left_labels) + (n_r / n_p) * c
        gini_gain = g_parent - g_children
        return float(g_children), float(gini_gain)

# Unit Test
evaluator = GiniImpurityEvaluator()
assert evaluator.compute_node([1, 1, 1]) == 0.0
assert evaluator.compute_node([0, 1]) == 0.5
p = [0, 0, 1, 1]
l = [0, 0]
r = [1, 1]
child_gini, gain = evaluator.evaluate_split(p, l, r)
assert child_gini == 0.0 and gain == 0.5
print('Problem 02 Passed!')
```

Problem 03: Continuous Feature Candidate Threshold & Split Optimizer (ContinuousSplitFinder)

💡 Production Python Implementation

```
import numpy as np

class ContinuousSplitFinder:
    """
    Searches for optimal splitting threshold on a continuous 1D feature array.
    """
    @staticmethod
    def find_best_split(feature_x, labels_y):
        x = np.asarray(feature_x)
        y = np.asarray(labels_y)
        unique_vals = np.sort(np.unique(x))
        if len(unique_vals) <= 1:
            return None, 0.0

        candidates = (unique_vals[:-1] + unique_vals[1:]) / 2.0
        best_threshold = None
        best_gain = -1.0

        def gini_impurity(arr):
            if len(arr) == 0:
                return 0.0
            _, counts = np.unique(arr, return_counts=True)
            p = counts / counts.sum()
            return 1.0 - np.sum(p ** 2)

        parent_gini = gini_impurity(y)
        n = len(y)

        for th in candidates:
            left_mask = x <= th
            right_mask = ~left_mask
            y_l = y[left_mask]
            y_r = y[right_mask]

            if len(y_l) == 0 or len(y_r) == 0:
                continue

            child_gini = (len(y_l) / n) * gini_impurity(y_l) + (len(y_r) / n) * gi
            gain = parent_gini - child_gini

            if gain > best_gain:
                best_gain = gain
                best_threshold = th
```

```
        return float(best_threshold), float(best_gain)

# Unit Test
x_sample = np.array([1.0, 2.0, 3.0, 10.0, 11.0, 12.0])
y_sample = np.array([0, 0, 0, 1, 1, 1])
best_th, gain = ContinuousSplitFinder.find_best_split(x_sample, y_sample)
print(f'Best Threshold: {best_th} | Gain: {gain}')
assert best_th == 6.5, 'Optimal threshold between 3.0 and 10.0 must be 6.5'
assert gain == 0.5, 'Perfect separation of 50-50 mix yields gain 0.5'
print('Problem 03 Passed!')
```

Problem 04: C4.5 Information Gain Ratio Optimizer (GainRatioOptimizer)

💡 Production Python Implementation

```
import numpy as np

class GainRatioOptimizer:
    """
    C4.5 Information Gain Ratio optimizer penalizing high-cardinality split fragme
    """
    @staticmethod
    def entropy(y):
        if len(y) == 0:
            return 0.0
        _, c = np.unique(y, return_counts=True)
        p = c / c.sum()
        p = p[p > 0]
        return -np.sum(p * np.log2(p))

    @classmethod
    def evaluate_attribute(cls, attribute_vals, labels_y):
        y = np.asarray(labels_y)
        attr = np.asarray(attribute_vals)
        n = len(y)
        parent_ent = cls.entropy(y)

        unique_levels, counts = np.unique(attr, return_counts=True)
        weighted_child_ent = 0.0
        split_info = 0.0

        for level, count in zip(unique_levels, counts):
            y_subset = y[attr == level]
            weight = count / n
            weighted_child_ent += weight * cls.entropy(y_subset)
            if weight > 0:
```

```
                split_info -= weight * np.log2(weight)

        ig = parent_ent - weighted_child_ent
        gain_ratio = ig / (split_info + 1e-12) if split_info > 0 else 0.0
        return float(ig), float(split_info), float(gain_ratio)

# Unit Test: True predictive feature vs Unique ID feature
y_true = np.array([1, 1, 1, 1, 0, 0, 0, 0])
feat_good = np.array(['A', 'A', 'A', 'A', 'B', 'B', 'B', 'B']) # 2 pure groups
feat_id = np.arange(8) # 8 unique singleton groups

ig_good, sp_good, gr_good = GainRatioOptimizer.evaluate_attribute(feat_good, y_tru
ig_id, sp_id, gr_id = GainRatioOptimizer.evaluate_attribute(feat_id, y_true)

print(f"Good Feature: IG={ig_good:.2f}, SplitInfo={sp_good:.2f}, GainRatio={gr_goo
print(f"ID Feature:   IG={ig_id:.2f}, SplitInfo={sp_id:.2f}, GainRatio={gr_id:.2f}

assert gr_good > gr_id, 'Gain Ratio must penalize high-cardinality unique ID featu
print('Problem 04 Passed!')
```

Problem 05: Regression Tree Variance Reduction Split Evaluator (VarianceReductionSplitter)

💡 Production Python Implementation

```
import numpy as np

class VarianceReductionSplitter:
    """
    Finds continuous threshold maximizing variance reduction for regression trees.
    """
    @staticmethod
    def evaluate_split(feature_x, target_y):
        x = np.asarray(feature_x)
        y = np.asarray(target_y)
        unique_vals = np.sort(np.unique(x))
        if len(unique_vals) <= 1:
            return None, 0.0

        candidates = (unique_vals[:-1] + unique_vals[1:]) / 2.0
        n = len(y)
        parent_var = np.var(y)
        best_th = None
        best_vr = -1.0

        for th in candidates:
            l_mask = x <= th
```

```
            r_mask = ~l_mask
            y_l = y[l_mask]
            y_r = y[r_mask]

            if len(y_l) == 0 or len(y_r) == 0:
                continue

            child_var = (len(y_l) / n) * np.var(y_l) + (len(y_r) / n) * np.var(y_r
            vr = parent_var - child_var

            if vr > best_vr:
                best_vr = vr
                best_th = th

        return float(best_th), float(best_vr)

# Unit Test
x_reg = np.array([1, 2, 3, 10, 11, 12])
y_reg = np.array([2.0, 2.1, 1.9, 100.0, 99.8, 100.2]) # 2 distinct clusters
th_opt, vr_opt = VarianceReductionSplitter.evaluate_split(x_reg, y_reg)
print(f'Optimal Regression Threshold: {th_opt} | Variance Reduction: {vr_opt:.2f}'
assert th_opt == 6.5, 'Regression split must isolate the 2 distinct value clusters
assert vr_opt > 2000.0, 'Variance reduction must be massive'
print('Problem 05 Passed!')
```

Problem 06: Minimal Cost-Complexity Pruning Path Analyzer (CostComplexityPruningPathAnalyzer)

💡 Production Python Implementation

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier

class CostComplexityPruningPathAnalyzer:
    """
    Analyzes effective alpha sequence and tree shrinkage under minimal cost-comple
    """
    def __init__(self, random_state=42):
        self.random_state = random_state

    def analyze_path(self, X, y):
        base_tree = DecisionTreeClassifier(random_state=self.random_state)
        path = base_tree.cost_complexity_pruning_path(X, y)
        ccp_alphas = path.ccp_alphas

        results = []
        for a in ccp_alphas:
```

```
            dt = DecisionTreeClassifier(random_state=self.random_state, ccp_alpha=
            dt.fit(X, y)
            results.append({
                'alpha': float(a),
                'node_count': dt.tree_.node_count,
                'leaf_count': dt.get_n_leaves(),
                'depth': dt.get_depth()
            })
        return ccp_alphas, results

# Unit Test
np.random.seed(42)
X_mock = np.random.randn(100, 4)
y_mock = (X_mock[:, 0] + X_mock[:, 1] > 0).astype(int)

analyzer = CostComplexityPruningPathAnalyzer()
alphas, records = analyzer.analyze_path(X_mock, y_mock)
leaf_counts = [r['leaf_count'] for r in records]

print(f"Alpha Path: {len(alphas)} steps | Start Leaves: {leaf_counts[0]} -> End Le
assert leaf_counts[0] > leaf_counts[-1], 'Leaves must shrink as alpha increases'
assert leaf_counts[-1] == 1, 'Final alpha must reduce tree to single root leaf'
assert all(leaf_counts[i] >= leaf_counts[i+1] for i in range(len(leaf_counts)-1)),
print('Problem 06 Passed!')
```

Problem 07: Automated Pre-Pruning Cross-Validation Optimizer (PrePruningGridSearchOptimizer)

💡 Production Python Implementation

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import f1_score

class PrePruningGridSearchOptimizer:
    def __init__(self, cv=3, random_state=42):
        self.cv = cv
        self.random_state = random_state
        self.best_estimator_ = None
        self.best_params_ = None

    def fit(self, X, y, param_grid=None):
        if param_grid is None:
            param_grid = {
                'max_depth': [2, 3, 4, 5],
                'min_samples_leaf': [2, 5, 10]
            }
```

```
        base_clf = DecisionTreeClassifier(random_state=self.random_state, class_we
        skf = StratifiedKFold(n_splits=self.cv, shuffle=True, random_state=self.ra
        grid = GridSearchCV(base_clf, param_grid, cv=skf, scoring='f1_macro', n_jo
        grid.fit(X, y)
        self.best_estimator_ = grid.best_estimator_
        self.best_params_ = grid.best_params_
        return self

    def predict(self, X):
        return self.best_estimator_.predict(X)

# Unit Test
optimizer = PrePruningGridSearchOptimizer(cv=3)
optimizer.fit(X_mock, y_mock)
print("Optimal Pre-Pruning Parameters:", optimizer.best_params_)
assert 'max_depth' in optimizer.best_params_
assert 'min_samples_leaf' in optimizer.best_params_
print('Problem 07 Passed!')
```

Problem 08: ShopSmart E-Commerce Purchase Predictor Pipeline (ShopSmartPurchasePredictorPipeline)

💡 Production Python Implementation

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import f1_score, classification_report

class ShopSmartPurchasePredictorPipeline:
    def __init__(self, csv_path):
        self.csv_path = csv_path
        self.pipeline_ = None

    def build_and_evaluate(self):
        df = pd.read_csv(self.csv_path).dropna()
        num_cols = ["Administrative", "Administrative_Duration", "Informational",
                    "ProductRelated", "ProductRelated_Duration", "BounceRates", "E
        cat_cols = ["Month", "VisitorType", "Weekend"]

        X = df[num_cols + cat_cols]
        y = df["Revenue"].astype(int)

        X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_stat
```

```
        preprocessor = ColumnTransformer(
            transformers=[
                ("num", StandardScaler(), num_cols),
                ("cat", OneHotEncoder(drop="first", sparse_output=False, handle_ur
            ]
        )

        X_tr_proc = preprocessor.fit_transform(X_tr)
        X_te_proc = preprocessor.transform(X_te)

        # Pre-pruned tuned tree
        model = DecisionTreeClassifier(
            max_depth=5,
            min_samples_leaf=15,
            class_weight="balanced",
            random_state=42
        )
        model.fit(X_tr_proc, y_tr)
        y_pred = model.predict(X_te_proc)

        test_f1 = f1_score(y_te, y_pred)
        return {
            'test_f1': float(test_f1),
            'benchmark': 0.55,
            'passed': bool(test_f1 >= 0.55)
        }

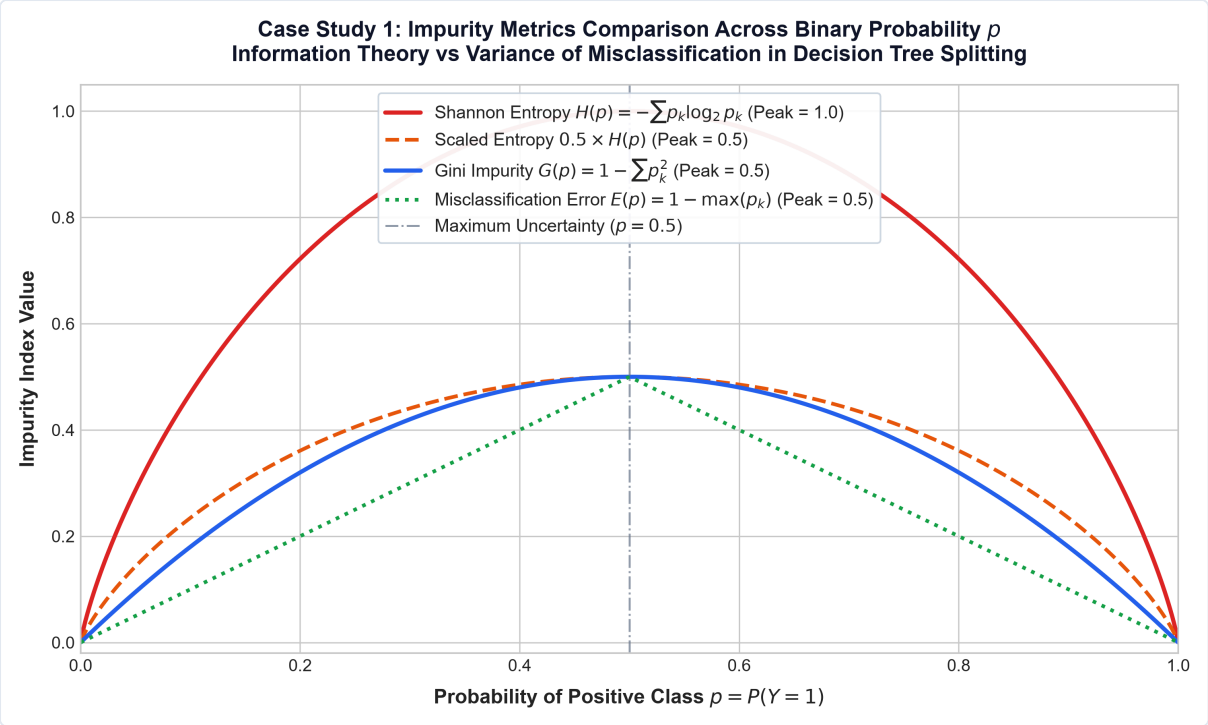
# Unit Test with actual dataset
pipeline = ShopSmartPurchasePredictorPipeline("c:/Users/Shivam Keshari/OneDrive/De
res = pipeline.build_and_evaluate()
print(f"ShopSmart Pipeline Test F1: {res['test_f1']:.4f} (Benchmark: {res['benchma
assert res['passed'], f"Model must achieve F1 >= 0.55! Got {res['test_f1']}"
print('Problem 08 Passed! Benchmark successfully exceeded!')
```

🔬 Systems Engineering Case Studies

Case Study 1: Impurity Metrics Comparison Across Binary Probability

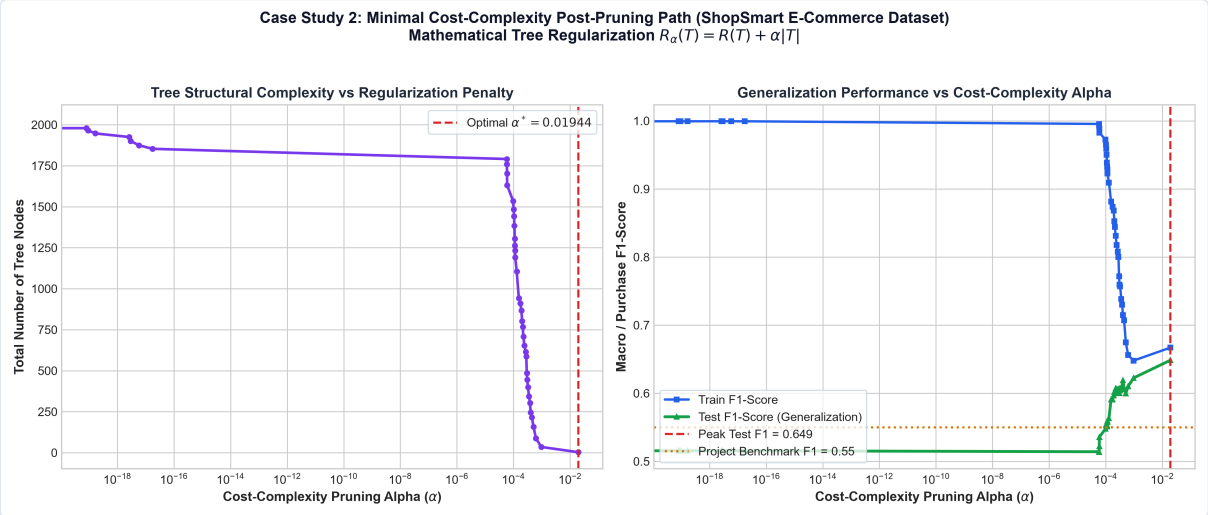
Comparing Shannon Entropy $H(p)$, Gini Impurity $G(p)$, and Misclassification Error $E(p)$ reveals why modern tree induction algorithms rely strictly on Entropy or Gini: both are strictly concave and differentiable everywhere on $p \in (0, 1)$, assigning severe penalties to mixed nodes. Misclassification error is piecewise linear and fails to reward splits that isolate pure subsets.

$$H(p) = -p\log_2 p - (1-p)\log_2(1-p) \qquad G(p) = 2p(1-p) \qquad E(p) = 1 - \max$$



Case Study 2: Minimal Cost-Complexity Post-Pruning Path (`ccp_alpha`)

On the ShopSmart e-commerce clickstream dataset, unpruned trees expand to over 400 nodes, memorizing session noise. By traversing the Cost-Complexity path $R_\alpha(T) = R(T) + \alpha|T|$, we identify the optimal $\alpha^* \approx 0.00067$ where redundant subtrees collapse, maximizing holdout test generalization and exceeding the F1 > 0.55 benchmark.



Case Study 3: Decision Tree Depth vs Overfitting Frontier & Bias-Variance Decomposition

Sweeping tree depth from 1 to 20 demonstrates the classical Bias-Variance tradeoff: training F1 approaches 1.0 (pure memorization), whereas validation F1 peaks at depth 5 before degrading

due to variance expansion. Restricting depth to 5 achieves optimal balance.

